

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 17

Sequences and methods



Università
Bocconi
MILANO

Objectives of the lesson

Learning how to access and how to handle:

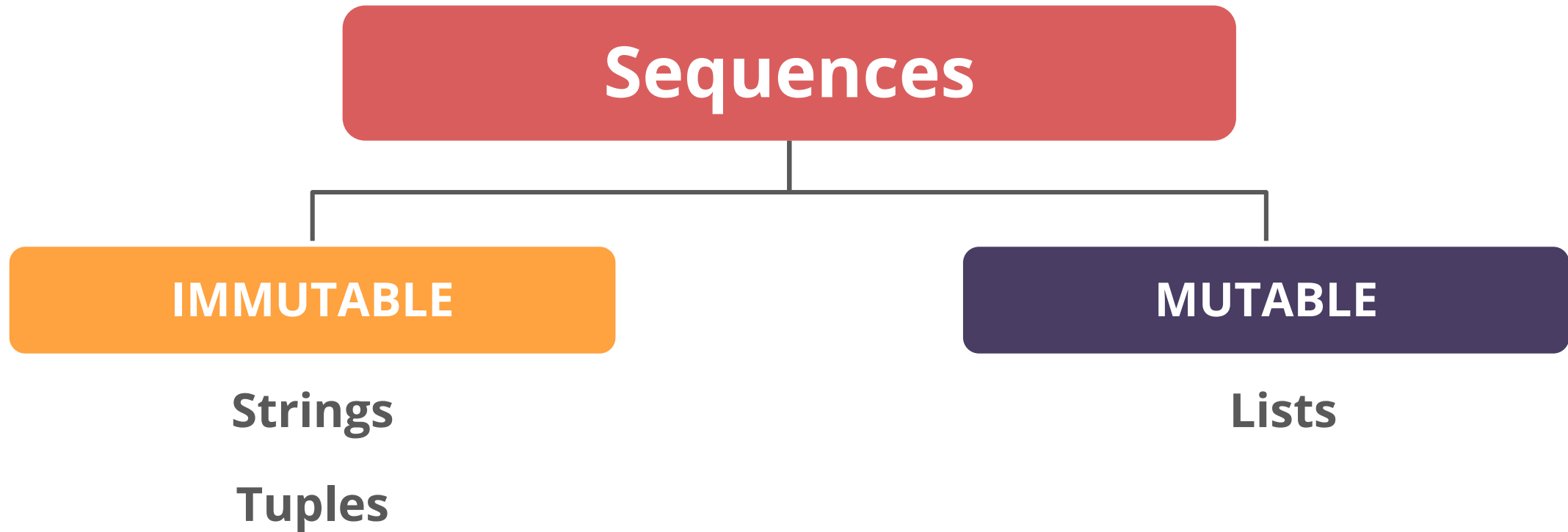
- Strings
- Lists
- Tuples



Sequences

- Sequences are objects that hold multiple items, stored one after the other
- There are different types of sequences, including strings, lists and tuples
- All sequences share the following characteristics:
 - they are iterable objects
 - they can be mutable or immutable
 - the position of each item is identified by a number (index)
 - they use many functions, methods and operations to access and work on the data

Mutability of sequences



Strings

- A string is a sequence of characters, and it can contain alphanumeric characters and symbols
- Strings are **immutable**, therefore an existing string cannot be modified, but it's possible to create a new string that is a variation of the original one (i.e. it overwrites the original string)

Example:

```
>>> univ = 'Bocconi'
```

```
>>> univ = 'UniBocconi'
```

Lists

- A list is a sequence of values (or elements) of any type, enclosed in square brackets and divided by commas:

```
[19, 30, 24, 28]  ['Bocconi', 'IULM', 'Bicocca']  [156, 'Via Roma', 'Milano', 3318488888, 15.36]
```

- Lists can be assigned to **variables**:

```
grades = [19, 30, 24, 28]
```

- A list with no elements is called **empty list**: `[]`
- A list can be **nested** within another list: `['New York', 9.5, 2, [10, 20]]`
- To create a list, it is possible to use the built-in function **list**:

```
>>> list('Bocconi')  
['B', 'o', 'c', 'c', 'o', 'n', 'i']
```

```
>>> list(range(4))  
[0, 1, 2, 3]
```



Tuples

- A tuple is a sequence very similar to a list. The elements of a tuple can be of any type, they are comma-separated and, usually, they are enclosed in a set of parentheses (although the use of parentheses is not necessary):

```
>>> t = ('b', 'o', 'c', 'c', 'o', 'n', 'i')
```

- To create a tuple, we can use the built-in function **tuple**:

```
>>> tuple('Bocconi')  
('B', 'o', 'c', 'c', 'o', 'n', 'i')
```

```
>>> tuple(range(4))  
(0, 1, 2, 3)
```

- Tuples are immutable, so it is not possible to modify an existing tuple, but it's possible to create a new tuple that is a variation on the original one

Operators and functions of sequences

Operator/function	Description
+	Concatenates/merges two sequences
*	Repeats a sequence (i.e. it creates more copies of a sequence and merges them)
in	Returns <i>True</i> if an element is found in a sequence, otherwise it returns <i>False</i>
len(seq)	Returns the number of elements contained in the sequence
max(seq) and min(seq)	Returns the largest/smallest value in a sequence consisting only of strings or only of numbers
sorted(seq)	Returns a new list with the elements sorted in ascending order (<i>seq</i> must consist only of strings or only of numbers)
sum(seq)	Sums the elements of the sequence (numbers only)

Indexing

- The position of each element within a sequence is identified by an integer, called index
- Indexing allows access to the individual elements of a sequence, using the following syntax: **sequence [index]**
- The index of the first element on the left is equal to **0**, while the index of the last element of a sequence of **n** elements is equal to **n-1**
- It is possible to use indices with negative values, in this case the count starts from the end of the sequence

LEFT TO RIGHT

0	1	2	3	4	5	6	7	8	9	10	11	12
P	Y	T	H	O	N		B	A	S	I	C	S
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

RIGHT TO LEFT



Using indices

- Accessing an element in a list:

```
>>> MyList = ['Milan', 'New York', 'London', 'Paris']
>>> print(MyList[2])
London
>>> MyList[2] = 'Berlin'
>>> print(MyList)
['Milan', 'New York', 'Berlin', 'Paris']
```

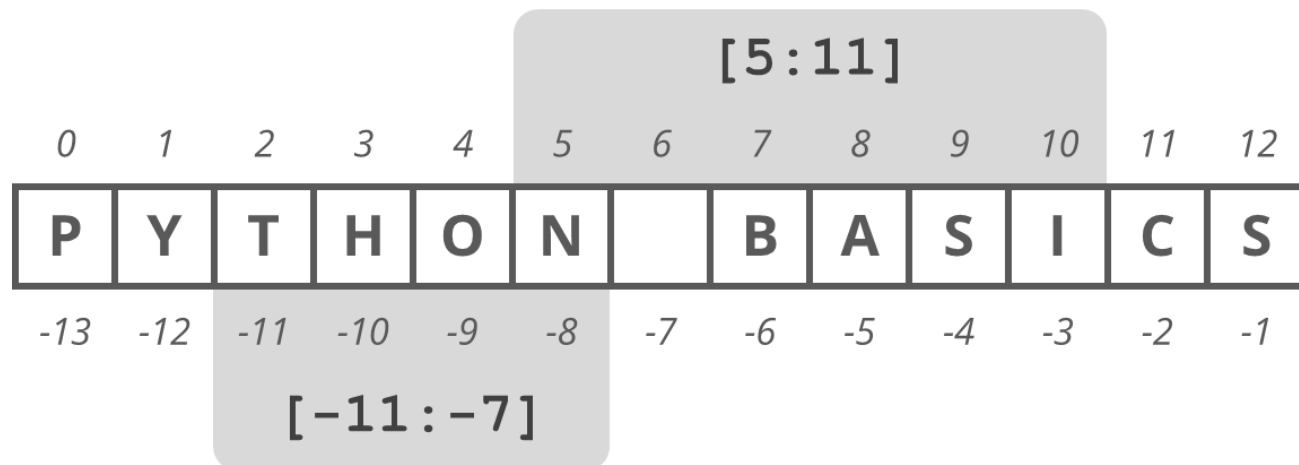
- Accessing an element in a string:

```
>>> MyString = 'Mikan'
>>> print(MyString[2])
k
>>> MyString[2] = 'l'
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    MyString[2] = 'l'
TypeError: 'str' object does not support item assignment
```



Slicing

- A segment or a portion of sequence is called **slice**.
- Slicing allows to select more than one element of a sequence using the syntax: **sequence [start: end: step]**
- Slicing returns all the elements of the sequence from the one with index *start* (included) to the one with index *end* (not included). Step is optional and indicates which subsequent indexes must be included after the first one.



Examples of slicing

- When *start* is omitted, the slice starts from the first element; when *end* is omitted, the slicing ends on the last element. By omitting both, the entire sequence is selected.
- Consider the string 'Computer viruses are an urban legend' stored in the variable *MyQuote*. Here are some examples of slicing:

Command	Meaning	Result
MyQuote[6:12]	Select all the elements from the one with index 6 to the one with index 11	'er vir '
MyQuote[:7]	Select all the elements from the first one up to the one with index 6	'Compute'
MyQuote[3:20:2]	Selects an element every two starting from the one with index 3 up to the one with index 19	'ptrvrssae'
MyQuote[:15:3]	Selects an element every three starting from the first one up to the one with index 14	'Cpevu'

Traversing sequences

- It is often necessary to traverse, one by one, all elements of a sequence
- Sequences can be traversed by **using their elements**:

```
teams=['Alpha','Beta','Epsilon','Gamma']  
for elem in teams:  
    print(elem)
```

- Sequences can also be traversed by **using their indices**:

```
teams=['Alpha','Beta','Epsilon','Gamma']  
for i in range(len(teams)):  
    elem = teams[i]  
    print(elem)
```

```
teams=['Alpha','Beta','Epsilon','Gamma']  
i = 0  
while i < len(teams):  
    elem = teams[i]  
    print(elem)  
    i = i + 1
```

Methods

- Strings, lists and tuples have special functions, called **methods**
- The **syntax of functions** needs the sequence (or the name of the variable containing a sequence) as an argument of the function
- The **syntax of methods** runs the method directly from the sequence (or from the name of a variable containing a sequence), using the following syntax:

`sequence.method(arguments)`

Methods of strings

Method	Description
<code>.upper()</code>	Returns a copy of the string with all characters converted to uppercase. Any character that is already uppercase is not modified.
<code>.lower()</code>	Returns a copy of the string with all characters converted to lowercase. Any character that is already lowercase is not modified.
<code>.find(sub)</code>	Search in the string for the substring specified by the <i>sub</i> argument and returns the index of the first occurrence found.
<code>.startswith(prefix)</code>	Returns <i>True</i> if the string starts with the string specified in the <i>prefix</i> argument, otherwise it returns <i>False</i> .
<code>.endswith(suffix)</code>	Returns <i>True</i> if the string ends with the string specified in the <i>suffix</i> argument, otherwise it returns <i>False</i> .
<code>.count(sub)</code>	Returns the number of occurrences of the substring <i>sub</i> in the string.
<code>.split()</code>	Splits a string into words and returns a list of strings, using by default the space character as a separator between words.
<code>.join(iterable)</code>	Use the string as separator to concatenate all the elements of iterable (which has to be an iterable object containing only strings).



Methods of lists

Method	Description
<code>.append(element)</code>	Appends <i>element</i> at the end of the list
<code>.insert(index, element)</code>	Inserts <i>element</i> in the position specified by the <i>index</i> parameter
<code>.remove(element)</code>	Searches for <i>element</i> in the list and removes its first occurrence
<code>.pop(index)</code>	Removes and returns the element corresponding to the given index. If <i>index</i> is omitted, it removes and returns the last element of the list
<code>.index(element)</code>	Searches for <i>element</i> within the list and returns the index of its first occurrence
<code>.count(element)</code>	Counts how many times <i>element</i> is found in the list
<code>.sort()</code>	Sorts the elements of the list in ascending order (by default)
<code>.reverse()</code>	Reverses the current order of the list elements



Methods of tuples

- The tuple methods are only a few because tuples are immutable
- It is often useful to convert a tuple into a list, work on the list elements, then convert it back into a tuple

Method	Description
<code>.index(element)</code>	Searches for <i>element</i> within the tuple and returns the index of its first occurrence.
<code>.count(element)</code>	Returns the number of occurrences of <i>element</i> in the tuple.

Exercise 1

Create a new Python file, named **Exercise 1.py**, containing a function named **random_list** which creates a list of random numbers. The function must:

- receive an integer representing the number of elements of the list, as mandatory parameter
- create a list of random integers between 1 and 100
- return the list of random numbers

Now create a copy of the previous function, name it **random_list2** and modify it so that the list of random numbers is created without duplicate values.

Exercise 2

Create a new Python file, named **Exercise 2.py**, containing a function named **FirstLast** which extracts the first and the last character of a sentence received as argument. In particular, the function must:

- receive a string, representing a sentence, as mandatory argument
- return a text string which shows, in uppercase, the first and the last character of the sentence (e.g. "First character: X, last character: Y")

Create a second function, named **FirstLast2**, that:

- receives a string as mandatory argument and two integers, representing two indices, as optional arguments (default values: 0 and 1)
- returns a string that displays, in uppercase, all the characters between the two indices defined by the optional parameters, both inclusive (for example, if the optional arguments are 1 and 5, the function must display the characters from index 1 up to index 5, both included)

Exercise 2

Create a third function, named **FirstLast3**, to extract the first and the last word of a sentence. In particular, the function must:

- receive a string, which represents a sentence, as mandatory argument
- return a string which shows the first and the last word of the sentence, written in uppercase (e.g. "First word: AAA, last word: BBB")

Tools to learn with today's lecture

- How to handle and use strings
- How to handle and use lists
- How to handle and use tuples

Book references

Understanding Python:

Chapter 8 (except 8.9 and 8.14)

Python workbook:

Exercises of Units 8, 9